

```
In [ ]: #3.1 Decision Tree классификация.
```

```
In [16]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
```

```
In [ ]: ## 1. Загрузка и обработка датасета.
Датасет, его загрузка и обработка как в заданиях 1.1, 2.1.
```

```
In [17]: df = pd.read_csv('C:/Users/Elizaveta/Downloads/magic_gamma/telescope_data.csv')
pd.set_option('display.max_columns', None)
pd.set_option('display.width', None)
df.head(5)
df.info()
print("размер", df.shape)
print("названия столбцов", list(df.columns))
print("уникальные значения в class", df['class'].unique())
print("\nраспределение классов")
print(df['class'].value_counts())
print("доли классов")
print(df['class'].value_counts(normalize=True))
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19020 entries, 0 to 19019
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Unnamed: 0   19020 non-null  int64
1   fLength      19020 non-null  float64
2   fWidth       19020 non-null  float64
3   fSize        19020 non-null  float64
4   fConc        19020 non-null  float64
5   fConc1       19020 non-null  float64
6   fAsym        19020 non-null  float64
7   fM3Long      19020 non-null  float64
8   fM3Trans     19020 non-null  float64
9   fAlpha       19020 non-null  float64
10  fDist        19020 non-null  float64
11  class        19020 non-null  object
dtypes: float64(10), int64(1), object(1)
memory usage: 1.7+ MB
размер (19020, 12)
названия столбцов ['Unnamed: 0', 'fLength', 'fWidth', 'fSize', 'fConc', 'fConc1',
'fAsym', 'fM3Long', 'fM3Trans', 'fAlpha', 'fDist', 'class']
уникальные значения в class ['g' 'h']
```

```
распределение классов
class
g    12332
h     6688
Name: count, dtype: int64
```

```
доли классов
class
g    0.64837
h    0.35163
Name: proportion, dtype: float64
```

```
In [19]: df['class']=df['class'].replace({'g': 1, 'h': 0})
print("first 10 values", df['class'].head(10))
print("unique values", df['class'].unique())
print("value counts", df['class'].value_counts())
print("type of data in class", df['class'].dtype)
print("first value", repr(df['class'].iloc[0]))
print("NaN", df['class'].isna().sum())
df=df.drop('Unnamed: 0', axis=1)
```

```

first 10 values 0    1
1    1
2    1
3    1
4    1
5    1
6    1
7    1
8    1
9    1
Name: class, dtype: int64
unique values [1 0]
value counts class
1    12332
0     6688
Name: count, dtype: int64
type of data in class int64
first value np.int64(1)
NaN 0

```

C:\Users\Elizaveta\AppData\Local\Temp\ipykernel_24512\2620301820.py:1: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version. To retain the old behavior, explicitly call `result.infer\_objects(copy=False)`. To opt-in to the future behavior, set `pd.set\_option('future.no\_silent\_downcasting', True)`

```
df['class']=df['class'].replace({'g': 1, 'h': 0})
```

```

In [4]: X=df.drop('class', axis=1)
        Y=df['class']
        print("size of X", {X.shape})
        print("size of Y", {Y.shape})
        X_train, X_test, Y_train, Y_test = train_test_split(
            X, Y,
            test_size=0.3,
            random_state = 42,
            stratify = Y)
        print(f"size of X_train", {X_train.shape})
        print(f"size of X_test", {X_test.shape})
        print(f"size of Y_train", {Y_train.shape})
        print(f"size of Y_test", {Y_test.shape})
        print("balance of classes:")
        print(f"start data: 1={Y.mean():.3f}, 0={1-Y.mean():.3f}")
        print(f"train data: 1={Y_train.mean():.3f}, 0={1-Y_train.mean():.3f}")
        print(f"start data: 1={Y_test.mean():.3f}, 0={1-Y_test.mean():.3f}")

```

```

size of X {(19020, 10)}
size of Y {(19020,)}
size of X_train {(13314, 10)}
size of X_test {(5706, 10)}
size of Y_train {(13314,)}
size of Y_test {(5706,)}
balance of classes:
start data: 1=0.648, 0=0.352
train data: 1=0.648, 0=0.352
start data: 1=0.648, 0=0.352

```

```

In [ ]: ##2. Создание бейзлайна.
        Обучение модели Logisticregression из библиотеки sklearn.
        F1 учитывает оба класса и их ошибки, показывает точность (сколько из отнесённых
        и полноту (сколько найдено настоящих объектов класса) классификации. В случае, е
        при высоком Accurasy, F1 покажет это.

```

Accuracy показывает, какую долю объектов модель классифицировала правильно. В да поэтому эта метрика даст адекватную оценку.
 Обучающая выборка (train) используется для обучения модели.
 Тестовая выборка (test) используется для оценки качества обученной модели.
 Результат на бейзлайне: f1=0.8614, accuracy=0.8211.
 Получены достаточно хорошие результаты, но есть потенциал для улучшения. Эти дан отталкиваться в сторону улучшений, показывающей, чего можно достичь минимальными

```
In [6]: from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import f1_score, accuracy_score
DTree_classifier = DecisionTreeClassifier(random_state=42)
DTree_classifier.fit(X_train, Y_train)
Y_prediction = DTree_classifier.predict(X_test)
F1 = f1_score(Y_test, Y_prediction)
accuracy = accuracy_score(Y_test, Y_prediction)
print(f" F1={F1:.4f}, accuracy={accuracy:.4f}")
```

F1=0.8614, accuracy=0.8211

In []: *##3. Улучшение бейзлайна.*
 Гипотеза 1: Подбор гиперпараметров.
 Подбираем гиперпараметры с помощью RandomizedSearchCV. максимальная глубина деревьев для разделения узла min_samples_split определяет рост дерева, минимальное число индекс Джини или энтропия. выбирается 50 случайных комбинаций параметров для оцен подобранные параметры помогут избежать переобучения и повысить качество модели.
 Результат: {'criterion': 'entropy', 'max_depth': 10, 'min_samples_leaf': 2, 'min

```
In [7]: from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint
param_dist_clf = {
    'max_depth': [3, 5, 7, 10, 15, None],
    'min_samples_split': randint(2, 20),
    'min_samples_leaf': randint(1, 10),
    'criterion': ['gini', 'entropy']
}
random_clf = RandomizedSearchCV(
    DecisionTreeClassifier(random_state=42),
    param_distributions=param_dist_clf,
    n_iter=50,
    cv=5,
    scoring='f1',
    random_state=42,
    n_jobs=-1
)
random_clf.fit(X_train, Y_train)
print("Лучшие параметры:", random_clf.best_params_)
```

Лучшие параметры: {'criterion': 'entropy', 'max_depth': 10, 'min_samples_leaf': 2, 'min_samples_split': 7}

Тестирование модели на полученных гиперпараметрах. Результаты: F1 = 0.8904, accuracy = 0.8523. Улучшение показало хорошие результаты, (f1 +1%, accuracy +1%).
 Ограничение глубины предотвращает переобучение, минимальные размеры узла и листа также способствуют обобщению. Критерий entropy позволил точнее ра

```
In [9]: best_tree_clf = DecisionTreeClassifier(
    criterion='entropy',
    max_depth=10,
    min_samples_leaf=2,
```

```

        min_samples_split=7,
        random_state=42
    )
    best_tree_clf.fit(X_train, Y_train)
    Y_pred_clf = best_tree_clf.predict(X_test)
    f1_clf = f1_score(Y_test, Y_pred_clf)
    accuracy_clf = accuracy_score(Y_test, Y_pred_clf)
    print(f"Улучшенный бейзлайн: F1 = {f1_clf:.4f}, accuracy = {accuracy_clf:.4f}")

```

Улучшенный бейзлайн: F1 = 0.8904, accuracy = 0.8523

In []: *## 4. Имплементация алгоритма машинного обучения.*
 Был создан собственный алгоритм решающего дерева MyDecisionTreeClassifier. Состоит из методов:

- `__init__` - задание гиперпараметров.
- `fit` - обработка входных данных, запуск построения рекурсивного дерева.
- `_build_tree` - рекурсивно строит дерево.
- `_best_split` - ищет лучшее разбиение.
- `_impurity` - вычисляет меру неопределённости для массива меток.
- `predict` - формирование предсказаний.

```

In [10]: class Node:
    def __init__(self, feature=None, threshold=None, left=None, right=None, proba=None):
        self.feature = feature
        self.threshold = threshold
        self.left = left
        self.right = right
        self.proba = proba

class MyDecisionTreeClassifier:
    def __init__(self, max_depth=None, min_samples_split=2, min_samples_leaf=1, criterion='entropy'):
        self.max_depth = max_depth
        self.min_samples_split = min_samples_split
        self.min_samples_leaf = min_samples_leaf
        self.criterion = criterion.lower()
        self.root = None
        self.n_classes_ = None
        self.n_features_ = None

    def fit(self, X, y):
        X = np.array(X)
        y = np.array(y).astype(int)
        self.n_classes_ = len(np.unique(y))
        self.n_features_ = X.shape[1]
        self.root = self._build_tree(X, y, depth=0)
        return self

    def _build_tree(self, X, y, depth):
        n_samples, n_features = X.shape
        n_classes = len(np.unique(y))

        if (depth == self.max_depth) or (n_samples < self.min_samples_split) or (n_classes < self.min_samples_leaf):
            proba = np.bincount(y, minlength=self.n_classes_) / len(y)
            return Node(proba=proba)

        best_feat, best_thresh, best_gain = self._best_split(X, y)

        if best_feat is None:
            proba = np.bincount(y, minlength=self.n_classes_) / len(y)
            return Node(proba=proba)

        left_mask = X[:, best_feat] <= best_thresh
        right_mask = ~left_mask
        left_child = self._build_tree(X[left_mask], y[left_mask], depth + 1)
        right_child = self._build_tree(X[right_mask], y[right_mask], depth + 1)
        return Node(feature=best_feat, threshold=best_thresh, left=left_child, right=right_child, proba=None)

    def _best_split(self, X, y):

```

```

best_gain = -float('inf')
best_feature = None
best_threshold = None
current_impurity = self._impurity(y)
for feature in range(self.n_features_):
    thresholds = np.unique(X[:, feature])
    for thresh in thresholds:
        left_mask = X[:, feature] <= thresh
        right_mask = ~left_mask
        if np.sum(left_mask) < self.min_samples_leaf or np.sum(right_mas
            continue
        left_impurity = self._impurity(y[left_mask])
        right_impurity = self._impurity(y[right_mask])
        n_left = np.sum(left_mask)
        n_right = np.sum(right_mask)
        n_total = n_left + n_right
        gain = current_impurity - (n_left / n_total) * left_impurity - (
        if gain > best_gain:
            best_gain = gain
            best_feature = feature
            best_threshold = thresh
    if best_gain <= 0:
        return None, None, None
    return best_feature, best_threshold, best_gain
def _impurity(self, y):
    if len(y) == 0:
        return 0
    probs = np.bincount(y, minlength=self.n_classes_) / len(y)
    if self.criterion == 'gini':
        return 1 - np.sum(probs ** 2)
    else:
        probs = probs[probs > 0]
        return -np.sum(probs * np.log2(probs))
def predict_proba(self, X):
    X = np.array(X)
    probas = []
    for x in X:
        node = self.root
        while node.proba is None:
            if x[node.feature] <= node.threshold:
                node = node.left
            else:
                node = node.right
        probas.append(node.proba)
    return np.array(probas)

def predict(self, X):
    proba = self.predict_proba(X)
    return np.argmax(proba, axis=1)

```

In [12]: Тестирование собственной реализации решающего дерева с подобранными ранее гиперп
F1 = 0.8899, accuracy = 0.8519
Полученные значения метрик идентичны значениям для библиотечной модели, это гоов

Cell In[12], line 1

Тестирование собственной реализации решающего дерева с подобранными ранее гип
ерпараметрами .Результаты:

^

SyntaxError: invalid syntax

```
In [11]: my_tree_clf = MyDecisionTreeClassifier(max_depth=10, min_samples_split=7, min_sa
my_tree_clf.fit(X_train, Y_train)
Y_pred_clf = my_tree_clf.predict(X_test)
f1 = f1_score(Y_test, Y_pred_clf)
accuracy = accuracy_score(Y_test, Y_pred_clf)
print(f"my tree: F1 = {f1:.4f}, accuracy = {accuracy:.4f}")
```

my tree: F1 = 0.8899, accuracy = 0.8519

In []: Использование балансировки классов. Параметр `class_weight='balanced'` автоматически позволяет модели уделять больше внимания меньшему классу. Результаты: F1=0.8828, Полученные значения говорят о том, что небольшой дисбаланс классов не влияет на

```
In [14]: tree_balanced = DecisionTreeClassifier(
    max_depth=10,
    min_samples_split=7,
    min_samples_leaf=2,
    criterion='entropy',
    class_weight='balanced',
    random_state=42
)
tree_balanced.fit(X_train, Y_train)
y_pred = tree_balanced.predict(X_test)
f1_bal = f1_score(Y_test, y_pred)
accuracy_bal = accuracy_score(Y_test, y_pred)
print(f"class_weight='balanced': F1={f1_bal:.4f}, accuracy={accuracy_bal:.4f}")
```

class_weight='balanced': F1=0.8828, accuracy=0.8458

In []: Обрезка решающего дерева для улучшения обобщающей способности. Этот метод уменьшает. Помогает бороться с переобучением и улучшает качество модели. Создаём дерево с параметром `ccp_alpha` (коэффициент сложности) вычисляется соответствующее дерево и чтобы сократить время перебора, выполняется установка текущего `ccp_alpha` в модель с максимальным средним F1. Результаты:
Лучший `ccp_alpha`: 0.000781 (очень маленькое значение, дерево обрезается слегка)
После обрезки: F1=0.8860, ROC-AUC=0.8891
Получено незначительное снижение значений метрик, но такое дерево всё равно может

```
In [15]: from sklearn.model_selection import cross_val_score
tree = DecisionTreeClassifier(
    max_depth=10,
    min_samples_split=7,
    min_samples_leaf=2,
    criterion='entropy',
    random_state=42
)
path = tree.cost_complexity_pruning_path(X_train, Y_train)
ccp_alphas = path.ccp_alphas
train_scores = []
val_scores = []
for alpha in ccp_alphas[::-10]:
    tree.set_params(ccp_alpha=alpha)
    scores = cross_val_score(tree, X_train, Y_train, cv=5, scoring='f1')
    val_scores.append(scores.mean())
    tree.fit(X_train, Y_train)
    train_scores.append(f1_score(Y_train, tree.predict(X_train)))
best_alpha = ccp_alphas[::-10][np.argmax(val_scores)]
print(f"Лучший ccp_alpha: {best_alpha:.6f}")
tree_pruned = DecisionTreeClassifier(
    max_depth=10,
```

```

min_samples_split=7,
min_samples_leaf=2,
criterion='entropy',
ccp_alpha=best_alpha,
random_state=42
)
tree_pruned.fit(X_train, Y_train)
y_pred_pruned = tree_pruned.predict(X_test)
y_proba_pruned = tree_pruned.predict_proba(X_test)[:, 1]
f1_pruned = f1_score(Y_test, y_pred_pruned)
accuracy_pruned = accuracy_score(Y_test, y_pred_pruned)
print(f"После обрезки: F1={f1_pruned:.4f}, accuracy={accuracy_pruned:.4f}")

```

Лучший ccp_alpha: 0.000781

После обрезки: F1=0.8860, accuracy=0.8467

In []: *## Выводы:*

Базовая модель решающего дерева показала высокие метрики $f1=0.8614$, $accuracy=0.8$ излишней сложности. Для улучшения был проведён подбор гиперпараметров с помощью минимальное число объектов в узле и листе) и была найдена оптимальная комбинация $f1=0.8904$, $accuracy=0.8523$. Была создана собственная реализация решающего дерева и энтропии, а также всех найденных гиперпараметров. После обучения с теми же параметрами библиотечной реализации алгоритма ($f1=0.8519$, $accuracy=0.8899$). Балансировка классов дисбаланс классов умеренный. Обрезка дерева с подбором коэффициента сложности ($f1=0.8860$, $accuracy=0.8467$).